



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 09

NOMBRE COMPLETO: Alfonso D'Hernán Rodríguez Zuluaga

N° de Cuenta: 321561556

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 05

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 3 de mayo del 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Ejercicio 1:

Código en main:

```
struct nube_humo {
    glm::vec3 posicion;
    GLfloat salida;
    GLfloat tiempoVida;
};

std::vector<nube_humo> humos_izq;
std::vector<nube_humo> humos_der;
GLint smokeStart_timer = 800;
GLint actualLeft = 0;
GLfloat ultNube = 0.0f;
GLfloat intervalo = 0.1f;
GLfloat arm_throw = 0.0f;
GLfloat t = 0.0f;
GLfloat e = 0.5f;

if (smokeStart_timer < timer_fire && timer_fire < 5399) {
    if (timer_fire <= 3600) {
        if (now - ultNube >= intervalo) {
            nube_humo nuevoHumo_izq, nuevoHumo_der;
            nuevoHumo_izq.posicion = glm::vec3(-cos(-0.8f + timer_fire * 0.0085f) * 0.6f + 0.0f, -sin(-0.6 + timer_fire * 0.0085f) * 0.5f + 4.5f, 1.5f);
            nuevoHumo_der.posicion = glm::vec3(cos(-0.8f + timer_fire * 0.0085f) * 0.6f + 0.0f, sin(-0.6 + timer_fire * 0.0085f) * 0.5f + 4.5f, 1.5f);
            nuevoHumo_izq.salida = nuevoHumo_der.salida = now;
            nuevoHumo_izq.tiempoVida = nuevoHumo_der.tiempoVida = 4.0f;
            humos_izq.push_back(nuevoHumo_izq);
            humos_der.push_back(nuevoHumo_der);
            ultNube = now;
        }
    }

    if (humos_izq.size() != 0) {
        for (int i = 0; i < humos_izq.size(); i++) {
            if (now > (humos_izq[i].salida + humos_izq[i].tiempoVida)) {
                humos_izq.erase(humos_izq.begin() + i);
                i--;
                continue;
            }
        }
    }
}
```

```

if (humos_der.size() != 0) {
    for (int i = 0; i < humos_der.size(); i++) {
        if (now > (humos_der[i].salida + humos_der[i].tiempoVida)) {
            humos_der.erase(humos_der.begin() + i);
            i--;
            continue;
        }
    }

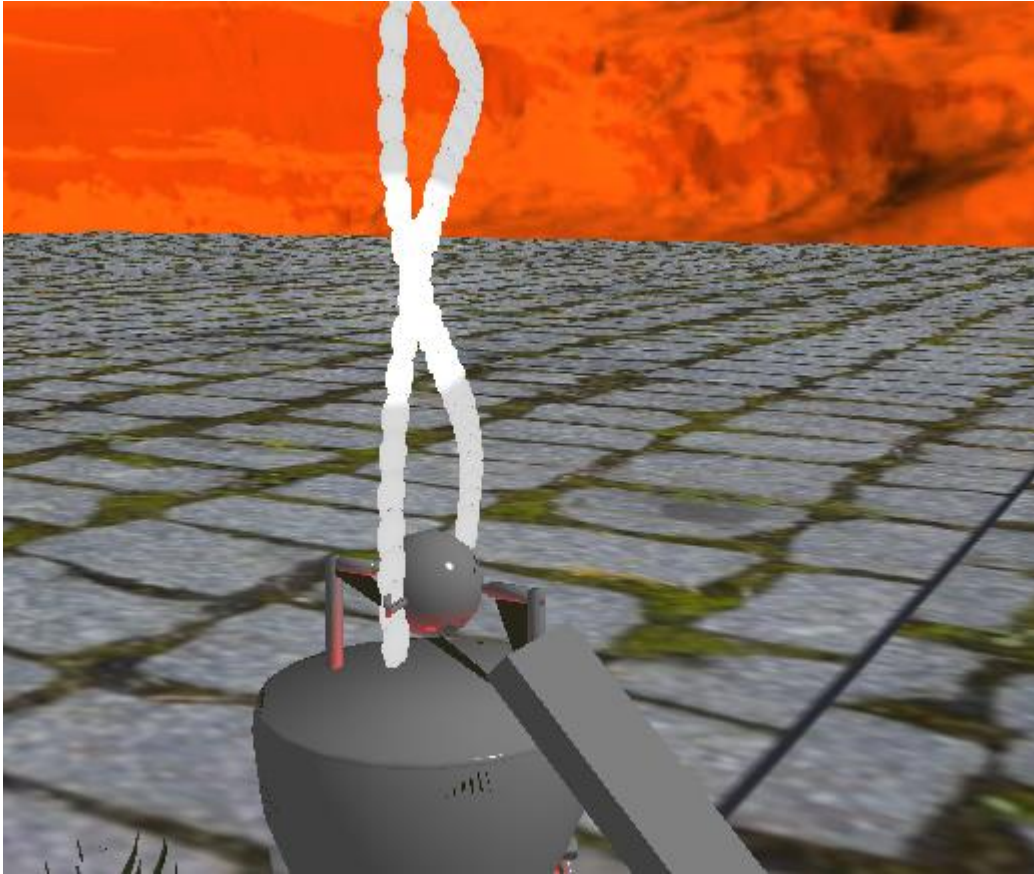
    glDepthMask(GL_FALSE);
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    for (const auto& nube : humos_izq) {
        GLfloat Y_act = nube.posicion.y;
        GLfloat X_act = nube.posicion.x;
        toffset = glm::vec2(toffsetnumerou, toffsetnumerov);
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(X_act, Y_act + ((now - nube.salida) * 2.0f), nube.posicion.z));
        model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
        model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
        glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        humo.UseTexture();
        Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
        meshList[5]->RenderMesh();
    }

    for (const auto& nube : humos_der) {
        toffset = glm::vec2(toffsetnumerou, toffsetnumerov);
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(nube.posicion.x, nube.posicion.y + ((now - nube.salida) * 2.0f), nube.posicion.z));
        model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
        model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
        glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        humo.UseTexture();
        Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
        meshList[5]->RenderMesh();
    }

    glDepthMask(GL_TRUE);
    glDisable(GL_BLEND);
}

```

Ejecución:



Ejercicio 2:

Código

```
void CrearDado()
```

```
    unsigned int cubo_indices[] = {  
        // front  
        0, 1, 2,  
        2, 3, 0,  
  
        // back  
        8, 9, 10,  
        10, 11, 8,  
  
        // left  
        12, 13, 14,  
        14, 15, 12,  
        // bottom  
        16, 17, 18,  
        18, 19, 16,  
        // top  
        20, 21, 22,  
        22, 23, 20,  
  
        // right  
        4, 5, 6,  
        6, 7, 4,
```

```
}; GLfloat cubo_vertices[] = {  
    // front  
    //x      y      z      S      T      NX      NY      NZ      //0  
    -0.5f, -0.5f, 0.5f, 0.26f, 0.34f, 0.0f, 0.0f, -1.0f, //0  
    0.5f, -0.5f, 0.5f, 0.49f, 0.34f, 0.0f, 0.0f, -1.0f, //1  
    0.5f, 0.5f, 0.5f, 0.49f, 0.66f, 0.0f, 0.0f, -1.0f, //2  
    -0.5f, 0.5f, 0.5f, 0.26f, 0.66f, 0.0f, 0.0f, -1.0f, //3  
    // right  
    //x      y      z      S      T  
    0.5f, -0.5f, 0.5f, 0.0f, 0.0f, -1.0f, 0.0f, 0.0f,  
    0.5f, -0.5f, -0.5f, 1.0f, 0.0f, -1.0f, 0.0f, 0.0f,  
    0.5f, 0.5f, -0.5f, 1.0f, 1.0f, -1.0f, 0.0f, 0.0f,  
    0.5f, 0.5f, 0.5f, 0.0f, 1.0f, -1.0f, 0.0f, 0.0f,  
    // back  
    -0.5f, -0.5f, -0.5f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f,  
    0.5f, -0.5f, -0.5f, 1.0f, 0.0f, 0.0f, 0.0f, 1.0f,  
    0.5f, 0.5f, -0.5f, 1.0f, 1.0f, 0.0f, 0.0f, 1.0f,  
    -0.5f, 0.5f, -0.5f, 0.0f, 1.0f, 0.0f, 0.0f, 1.0f,  
  
    // left  
    //x      y      z      S      T  
    -0.5f, -0.5f, -0.5f, 0.0f, 0.0f, 1.0f, 0.0f, 0.0f,  
    -0.5f, -0.5f, 0.5f, 1.0f, 0.0f, 1.0f, 0.0f, 0.0f,  
    -0.5f, 0.5f, 0.5f, 1.0f, 1.0f, 1.0f, 0.0f, 0.0f,  
    -0.5f, 0.5f, -0.5f, 0.0f, 1.0f, 1.0f, 0.0f, 0.0f,  
  
    // bottom  
    //x      y      z      S      T  
    -0.5f, -0.5f, 0.5f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f,  
    0.5f, -0.5f, 0.5f, 1.0f, 0.0f, 0.0f, 1.0f, 0.0f,  
    0.5f, -0.5f, -0.5f, 1.0f, 1.0f, 0.0f, 1.0f, 0.0f,  
    -0.5f, -0.5f, -0.5f, 0.0f, 1.0f, 0.0f, 1.0f, 0.0f,
```

```

//UP
//x      y      z      S      T
-0.5f, 0.5f, 0.5f, 0.0f, 0.0f, 0.0f, -1.0f, 0.0f,
0.5f, 0.5f, 0.5f, 1.0f, 0.0f, 0.0f, -1.0f, 0.0f,
0.5f, 0.5f, -0.5f, 1.0f, 1.0f, 0.0f, -1.0f, 0.0f,
-0.5f, 0.5f, -0.5f, 0.0f, 1.0f, 0.0f, -1.0f, 0.0f,
};

```

```

Mesh* dado = new Mesh();
dado->CreateMesh(cubo_vertices, cubo_indices, 192, 36);
meshList.push_back(dado);

```

```

model = modelaux;

model = glm::translate(model, glm::vec3(0.0f, 2.0f, 3.5f));
model = glm::rotate(model, -45.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.0f, 7.5f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[7]->RenderMesh();
model = modelaux;

model = glm::translate(model, glm::vec3(0.0f, -3.75f, 0.0f));
model = glm::rotate(model, arm_throw * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, -45.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::translate(model, glm::vec3(0.0f, -3.75f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.0f, 7.5f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[7]->RenderMesh();

model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -3.6f - y_parabolicThrow, 0.5f + z_parabolicThrow));
model = glm::scale(model, glm::vec3(0.0001f, 0.0001f, 0.0001f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
ball.RenderModel();

```

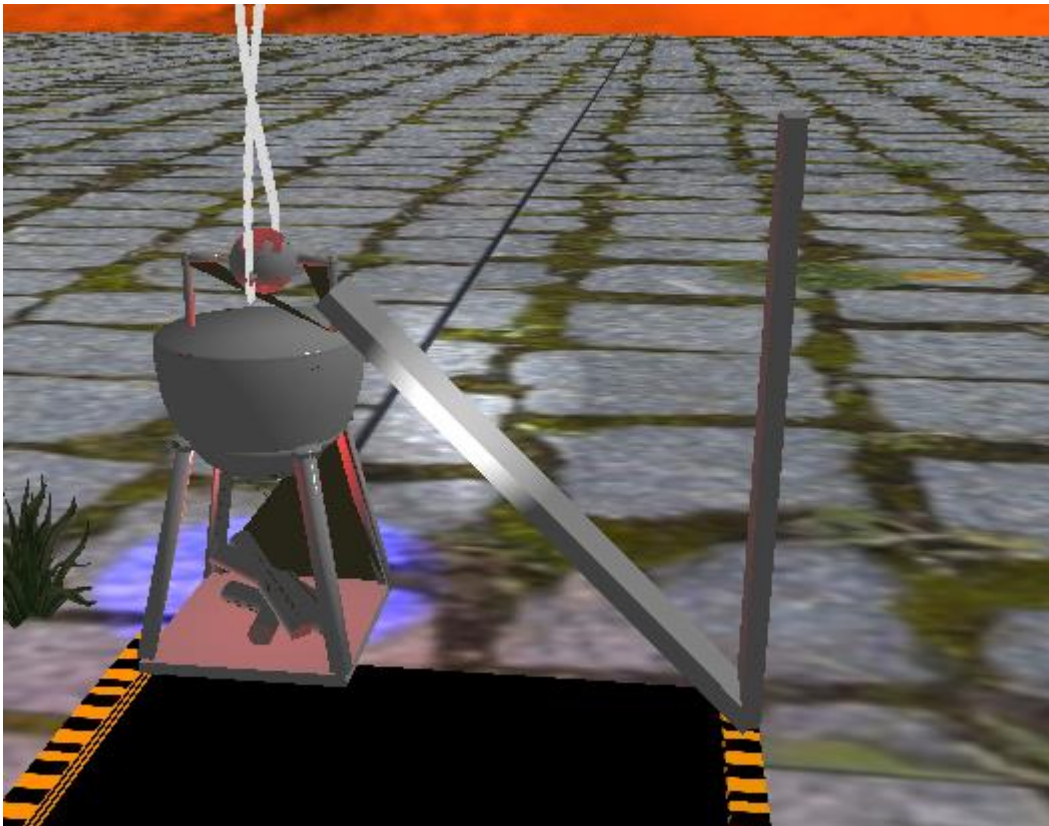
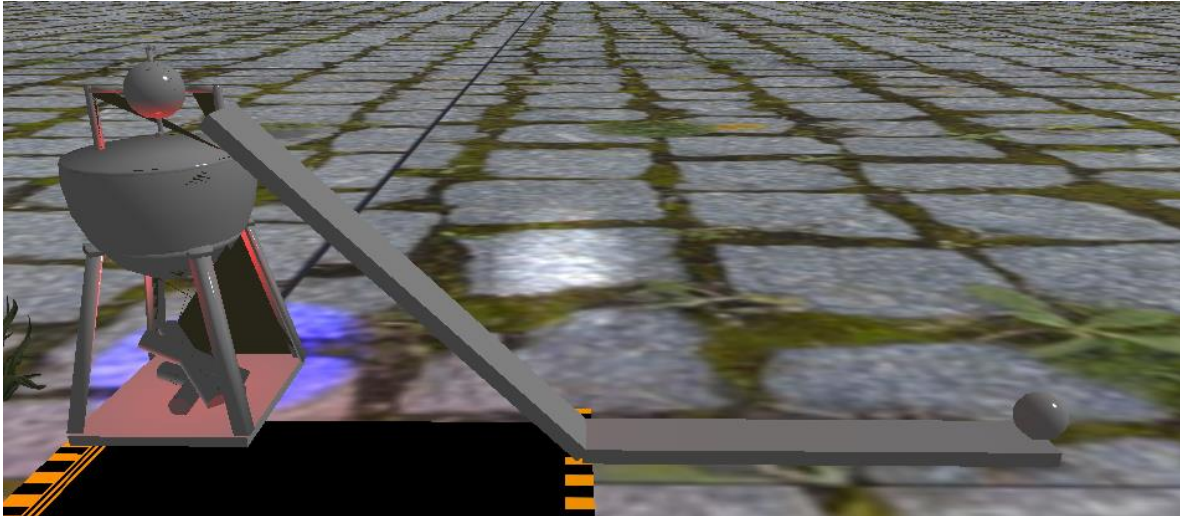
```

if (smokeStart_timer <= timer_fire && timer_fire < 3500) {
    arm_throw -= 0.005f;
}

if (smokeStart_timer <= timer_fire && timer_fire > 3500 && timer_fire < 3600) {
    arm_throw -= 0.765f;
}

```

Ejecución



Ejercicio 3:

Código:

```

model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -3.6f - y_parabolicThrow, 0.5f + z_parabolicThrow));
model = glm::scale(model, glm::vec3(0.0001f, 0.0001f, 0.0001f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
ball.RenderModel();

```

```

}

```

```

if (timer_fire > 3600 && timer_fire < 4000) {
    t = timer_fire - 3600;
    z_parabolicThrow = 0.2f * cos(45.0f * toRadians) * t;
    y_parabolicThrow = 0.2f * sin(45.0f * toRadians)*t - 0.5 * 0.0008f * t * t;
    y_save = y_parabolicThrow;
    z_save = z_parabolicThrow;
}

```

```

if (timer_fire > 4000 && timer_fire < 4375) {
    t = timer_fire - 4000;
    z_parabolicThrow = e*(0.2f * cos(45.0f * toRadians) * t) + z_save;
    y_parabolicThrow = e*(0.2f * sin(45.0f * toRadians)*t - 0.5 * 0.0008f * t * t) + y_save;
    if (timer_fire == 4374) {
        y_save = y_parabolicThrow + 0.5f;
        z_save = z_parabolicThrow;
    }
}

```

```

if (timer_fire > 4375 && timer_fire < 4750) {
    t = timer_fire - 4375;
    z_parabolicThrow = e*e*(0.2f * cos(45.0f * toRadians) * t) + z_save;
    y_parabolicThrow = e * e * (0.2f * sin(45.0f * toRadians)*t - 0.5 * 0.0008f * t * t) + y_save;
    if (timer_fire == 4749) {
        y_save = y_parabolicThrow;
        z_save = z_parabolicThrow;
    }
}

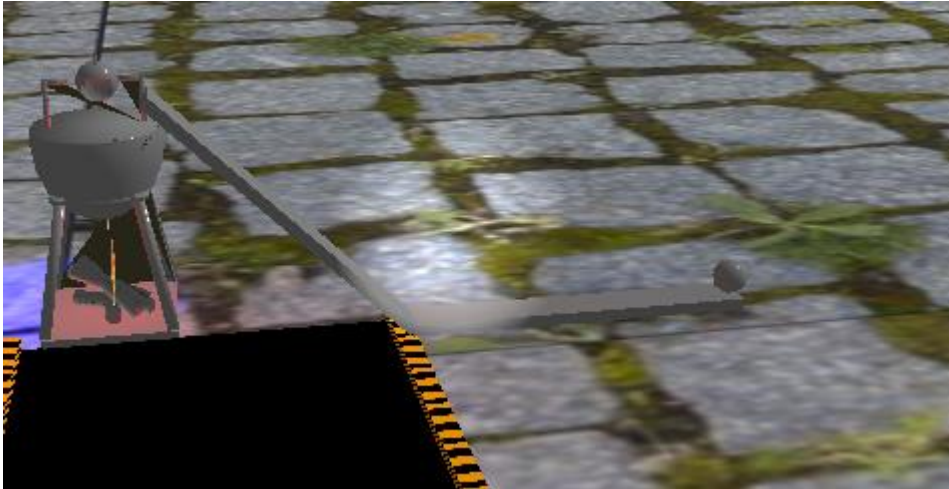
```

```

if (timer_fire > 5075 && timer_fire < 5400) {
    t = timer_fire - 5075;
    z_parabolicThrow = e * e * e * e * (0.2f * cos(45.0f * toRadians) * t) + z_save;
    y_parabolicThrow = -0.0005f * t * t + y_save;
}

```

Ejecución:



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

- Para las nubes de humo, la verdad es que pensé que instanciarlas utilizando colecciones sería extremadamente ineficaz pero al mismo tiempo mi implementación solo movía una sola textura a la vez. Sin saber qué hacer, recurrí a la IA, así fue nuestra conversación:

-
/*

PROMPT:

Animación:

- Simple o básica: Por banderas y condicionales (más de 1 transformación geométrica se ve modificada)

- Compleja: Por medio de funciones y algoritmos.

- Textura Animada

*/

// DEMÁS CÓDIGO...

how can I make several instances of the smoke appear?

- RESPUESTA:

To make several instances of smoke appear, the most efficient approach is to treat each smoke puff as an **object** within a **collection** (like a `std::vector`). This allows you to manage the individual life cycle, position, and transparency of each puff independently.

Here is a breakdown of how to implement a basic **Particle System** for your smoke.

1. Create a Smoke Structure

Define a struct to keep track of each individual puff's properties.

C++

```
struct SmokePuff {  
    glm::vec3 position;  
    float birthTime;  
    float lifeSpan; // How long it lasts in seconds  
    float offsetU;  
    float offsetV;  
};
```

```
std::vector<SmokePuff> smokeParticles;  
float lastPuffTime = 0.0f;  
float puffInterval = 0.2f; // Time between new puffs
```

2. Update Logic (Inside the while loop)

You need to generate new puffs at intervals and remove the old ones that have "died" (exceeded their lifespan).

C++

```
if (mainWindow.getPrendeFuego() == 1) {  
    // 1. Generate new particles  
    if (now - lastPuffTime > puffInterval) {  
        SmokePuff newPuff;  
        newPuff.position = glm::vec3(0.0f, 1.5f, 1.5f); // Start at the source
```

```

        newPuff.birthTime = now;
        newPuff.lifeSpan = 3.0f; // Lasts 3 seconds
        smokeParticles.push_back(newPuff);
        lastPuffTime = now;
    }
}

```

// 2. Update and Clean up particles

```

for (int i = 0; i < smokeParticles.size(); i++) {
    float age = now - smokeParticles[i].birthTime;

    // If it's too old, remove it
    if (age > smokeParticles[i].lifeSpan) {
        smokeParticles.erase(smokeParticles.begin() + i);
        i--; // Adjust index
        continue;
    }
}

```

3. Render Logic (Inside the while loop)

Iterate through your vector and draw each puff. Use the age variable to calculate its current height and horizontal "sway."

C++

```

glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
humo.UseTexture();

for (const auto& puff : smokeParticles) {
    float age = now - puff.birthTime;

```

```

// Calculate movement
float currentY = puff.position.y + (age * 1.5f); // Rises over time
float currentX = puff.position.x + sin(age * 3.0f) * 0.5f; // Sways over time

// Fade out as it gets older
float opacity = 1.0f - (age / puff.lifeSpan);

glUniform3f(uniformColor, opacity, opacity, opacity); // Simple way to fade if
shader supports it

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(currentX, currentY, puff.position.z));
model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));

// Scale: Puff gets bigger as it rises
float scale = 1.0f + age;
model = glm::scale(model, glm::vec3(scale, scale, scale));

// Send to shader
toffset = glm::vec2(0.0f, toffsetfuego); // Use your animated V offset
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));

meshList[5]->RenderMesh();
}

```

Key Improvements this provides:

- **Decoupling:** Each puff has its own birthTime, so their animations are offset. One is starting while another is halfway up and a third is disappearing.
- **Dynamic Scaling:** By multiplying the scale by age, the smoke looks like it's expanding as it dissipates.

- **Memory Management:** Using erase on the vector ensures you aren't trying to render thousands of invisible smoke puffs, keeping your FPS stable.

Pro-Tip: Transparency Sorting

For better visuals with GL_BLEND, you should ideally render your smoke particles **after** all opaque objects (like the car and track) and sort them from furthest to nearest to the camera. However, for a simple smoke trail, rendering them at the end of your loop usually suffices.

-PROMPT

Site your sources.

- RESPUESTA

My previous responses are based on a synthesis of industry-standard practices in computer graphics and software engineering. Specifically, the implementation logic for the `std::vector` particle system and the alternative approaches draw from the following foundational sources and paradigms:

1. Game Design Patterns

The "Update" and "Draw" logic used to manage particle lifetimes is a core architectural pattern known as the **Game Loop** and **Update Method**.

- **Source:** *Game Programming Patterns* by Robert Nystrom. This is the definitive text on why we decouple state updates (calculating age/position) from the rendering calls.

2. OpenGL Documentation and Tutorials

The method of rendering multiple instances by updating glUniform variables within a loop is the standard approach for introductory to intermediate OpenGL.

- **Source:** *LearnOpenGL.com* (Joey de Vries). This resource covers the transformation pipeline and the use of textures with transparency (blending) in depth.
- **Concept:** The specific use of `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)` is the standard implementation for alpha blending in the OpenGL Specification.

3. C++ Standard Template Library (STL)

The choice of `std::vector` over other containers is based on **Data-Oriented Design** principles.

- **Source:** *Effective C++* by Scott Meyers and lectures by Bjarne Stroustrup (creator of C++).

- **Rationale:** These sources explain that `std::vector` is preferred for high-performance loops because of **spatial locality**. Because vectors store data in contiguous memory, they take advantage of CPU caching, which is critical when you are iterating 60+ times per second.

4. Mathematics for Computer Graphics

The movement logic (using `sin()` for swaying and linear multipliers for rising) is rooted in **Kinematics** and **Trigonometry**.

- **Source:** *Mathematics for 3D Game Programming and Computer Science* by Eric Lengyel.
- **Concept:** This covers how to use parametric equations (where x and y are functions of t) to simulate physical phenomena like smoke or fire without a full physics engine.

5. Advanced Rendering Techniques

The alternative approaches mentioned (GPU Instancing and Compute Shaders) are documented in the **OpenGL SuperBible** and the **Vulkan/OpenGL API Specifications**.

- **Source:** *OpenGL SuperBible: Comprehensive Tutorial and Reference*.
- **Concept:** This provides the technical foundation for moving particle calculations from the CPU (the vector approach) to the GPU (Instancing/Compute approach) to handle higher complexity.

¿Cómo me cersioré de que la información fuera correcta?

No lo era en su totalidad, mientras que sí fundamentó la manera en la que llevé a cabo el primer ejercicio de la práctica, tenía algunas partes de la lógica que estaban mal, como la parte de dónde renderizar, o algunas restas lógicas que llevaban a que el programa no funcionase como hubiera querido.

Con todo esto, terminé modificando el código para que funcionase coherentemente.

¿Cuál fue el alcance de esta interacción con la IA?

Solamente el ejercicio 1, el resto de los ejercicios fueron enteramente de autoría mía. En su defecto, como la función de `CrearDado`, unas cuantas cosas se obtuvieron de prácticas pasadas.

- Para la cuestión de animación, fue mucho de prueba y error. Pero al final logré tener animaciones satisfactorias, me apoyé de calculadoras y de cuestiones teóricas como los valores horizontales y verticales de la circunferencia de un círculo, así como las fórmulas de tiro parabólico.

3.- Conclusión:

a. Los ejercicios del reporte: Complejidad, Explicación.

Me sorprendió que la complejidad se encontró más en entender el código que en implementar las funciones o los senos y cosenos como tal. La práctica fue fructífera y la disfruté bastante, en cierta medida debido a lo complejo de estos ejercicios.

Comentarios generales: Creo que fue la clase que mejor se ha explicado hasta el momento. Todo fue muy intuitivo.

b. Conclusión

Viendo que la práctica, a grandes rasgos, trataba sobre cómo hacer uso de las animaciones complejas para dar ilusiones de movimiento, así como el uso de fórmulas para la justificación de las mismas, puedo decir que la práctica se concluyó en su totalidad.

1. Bibliografía en formato APA

Fernández, J. L. (s. f.). *Parabolic Motion*. Fisicalab.
<https://www.fisicalab.com/en/section/parabolic-motion>

Google. (2024). *Gemini* (versión de mayo) [Modelo de lenguaje extenso].
<https://gemini.google.com/share/70ce798bb898>

Math Open Reference. (s. f.). *Parametric Equation of a Circle*.
<https://www.mathopenref.com/coordparamcircle.html>